

Documento Técnico

Rutas de Inspección y Puntos Críticos en Infraestructura de Campus

Sistema de modelado de redes de campus mediante grafos valorados

Curso: Estructuras de Datos

Proyecto: Proyecto III

Estudiante: Angie Mariela Alpizar Porras

Semestre: I Semestre 2026

Fecha de entrega: 21 de junio de 2026

1. Introducción

Este documento describe el diseño y la implementación del sistema Rutas de Inspección y Puntos Críticos en Infraestructura de Campus, un programa de consola escrito en C++ que modela la red física de un campus (edificios, subestaciones, intersecciones, pasarelas y los caminos que los conectan) como un grafo valorado y resuelve, sobre ese modelo, tres familias de problemas: el cálculo de rutas de inspección y de rutas alternativas ante cierres, la detección de vulnerabilidades estructurales (puentes y puntos de articulación) y la propuesta de un tendido mínimo de cobertura mediante el árbol de expansión mínima (MST).

El énfasis del proyecto no está solamente en programar los algoritmos clásicos de grafos, sino en aplicarlos a un problema realista de mantenimiento e inspección de infraestructura, donde cada algoritmo responde a una pregunta concreta del dominio: por dónde conviene ir, qué partes de la red son críticas, qué tan caro es conectar todo y qué pasa si una vía se cierra.

El programa funciona mediante una interfaz de menús de texto. Al iniciar carga dos archivos de datos en formato CSV (vértices y aristas) y luego permite ejecutar cada consulta de forma independiente. La salida es estrictamente textual, ya que el objetivo es demostrar el funcionamiento de las estructuras y los algoritmos, no construir una interfaz gráfica.

2. Descripción del problema

La infraestructura de un campus puede verse como una red: los lugares de interés son nodos y los caminos entre ellos son conexiones con un costo asociado. Sobre esa red se necesita responder preguntas típicas de operación y mantenimiento.

Cada conexión tiene una longitud y un límite de velocidad, de modo que su costo de recorrido se puede medir de dos formas: por distancia (la longitud) o por tiempo. Siguiendo la recomendación del enunciado, el peso por tiempo se calcula como $\text{tiempo} = \text{length} / \text{speed_limit}$. Además, una vía puede ser de un solo sentido (oneway) y puede o no permitir su cierre temporal (can_close).

Las preguntas que el sistema debe resolver son:

- Conectividad: ¿está toda la red conectada? ¿Cuántas partes separadas (componentes) tiene?
- Vulnerabilidad: ¿cuáles conexiones son puentes (su pérdida desconecta la red) y cuáles nodos son puntos de articulación?
- Ruteo: ¿cuál es la ruta más corta entre dos lugares, por tiempo o por distancia? ¿Existe una ruta alternativa que evite el tramo más crítico?
- Cobertura mínima: ¿cuál es el conjunto de conexiones de menor costo que mantiene todo conectado (MST), y cuánto se ahorra frente a usar toda la red?
- Simulación: si se cierran una o varias conexiones, ¿cuánto cambia la ruta (desvío en longitud y tiempo)?
- Reportes: ¿cuáles son los nodos y puentes más críticos del sistema?

3. Arquitectura del proyecto

El proyecto está organizado de forma modular. Cada estructura o responsabilidad vive en su propio archivo de cabecera (.h) y el programa principal se mantiene en main.cpp. Esta separación facilita

leer el código, ubicar cada algoritmo y reutilizar las estructuras básicas.

```
codigo/  
main.cpp      -> Menu principal y lectura de opciones  
Grafo.h       -> Clase Grafo: carga y todos los algoritmos  
Vertice.h     -> Estructura de un nodo (lugar del campus)  
Arista.h      -> Estructura de una conexion (camino)  
UnionFind.h   -> Conjuntos disjuntos (para Kruskal)  
vertices.csv  -> Datos de entrada: nodos  
aristas.csv   -> Datos de entrada: conexiones
```

Compilación (desde la carpeta codigo):

```
g++ -std=c++17 -o proyecto.exe main.cpp  
./proyecto.exe
```

El programa está estructurado como un conjunto de clases y estructuras, tal como pide el enunciado: la clase Grafo concentra el modelo y los algoritmos, la clase UnionFind da soporte a Kruskal, y las estructuras Vertice y Arista representan los datos. Todo el código incluye documentación interna (comentarios) explicando el propósito de cada método.

4. Modelado del grafo

4.1 Estructuras Vertice y Arista

Un Vertice guarda el identificador del nodo (id), su nombre, sus coordenadas x e y (usadas por la heurística de A*), su tipo de infraestructura y su prioridad. Una Arista guarda los dos extremos (u y v), la longitud, el límite de velocidad, si es de un solo sentido (oneway), si se puede cerrar (can_close) y un campo cerrada que indica si está deshabilitada en este momento (lo usa la simulación de cierres). La Arista ofrece el método tiempo() que devuelve length / speed_limit.

4.2 Lista de adyacencia

El grafo se representa con una lista de adyacencia: para cada nodo se guarda la lista de índices de las aristas que salen de él. Se eligió lista de adyacencia (y no matriz de adyacencia) porque la red es dispersa -cada lugar conecta solo con unos pocos vecinos-, por lo que recorrer solo las aristas existentes es más eficiente en tiempo y memoria que reservar una matriz de tamaño $V \times V$.

El sentido de las vías se modela al construir la adyacencia: una arista de doble vía se agrega a la lista de sus dos extremos, mientras que una vía de un solo sentido se agrega únicamente a la lista del nodo de origen. De esta forma los algoritmos de ruteo (Dijkstra y A*) respetan automáticamente el sentido de circulación. Para las consultas de conectividad, puentes y MST el grafo se trata como no dirigido, que es la interpretación natural de esos problemas.

Para localizar un nodo a partir de su id se usa una búsqueda lineal (buscarIndice). Es una decisión deliberada de simplicidad: mantiene el código claro y directo. Su costo es $O(V)$ y, por ser llamada dentro de varios algoritmos, es el principal factor de sobre costo del sistema; en la sección de análisis de complejidad se detalla su impacto.

5. Algoritmos implementados y justificación

Esta sección describe, módulo por módulo, el algoritmo utilizado y por qué es el adecuado para la pregunta que resuelve.

5.1 Carga y validación de datos

Los archivos `vertices.csv` y `aristas.csv` se leen línea por línea separando por comas. Se descartan encabezados y líneas vacías, se ignoran nodos sin id, nodos repetidos y aristas cuyos extremos no existan, avisando en pantalla. La validación resume la cantidad de nodos y aristas, calcula el grado de cada nodo y reporta los nodos aislados (sin ninguna conexión). La conectividad general se verifica con la opción de componentes conexas.

5.2 Componentes conexas (BFS y DFS)

Para saber si la red está toda conectada se recorren sus componentes con dos estrategias: BFS (recorrido en anchura, con una cola) y DFS (recorrido en profundidad, con recursión). Ambos parten de un nodo no visitado, marcan todo lo alcanzable y repiten hasta cubrir el grafo; el número de veces que hay que reiniciar es el número de componentes. Se implementan los dos porque el enunciado los pide explícitamente y porque permiten comparar las dos formas clásicas de explorar un grafo, que dan el mismo resultado de conectividad.

5.3 Puentes y puntos de articulación (Tarjan)

Una conexión es un puente si al quitarla aumenta el número de componentes; un nodo es un punto de articulación si al quitarlo (con sus aristas) ocurre lo mismo. Estos son justamente los elementos críticos de la red. Se detectan con el algoritmo de Tarjan, que en un solo recorrido DFS calcula, para cada nodo, su tiempo de descubrimiento (`disc`) y el menor tiempo alcanzable (`low`) a través de aristas de retorno. Con esos dos valores se decide: la arista hacia un hijo v es puente si $low[v] > disc[u]$, y u es articulación si $low[v] \geq disc[u]$ (con el caso especial de la raíz, que es articulación solo si tiene más de un hijo en el árbol DFS). Se eligió Tarjan porque encuentra todos los puentes y articulaciones en tiempo lineal $O(V + E)$, en lugar de probar a quitar cada arista o nodo y volver a verificar la conectividad, que sería mucho más costoso.

5.4 Ruteo: Dijkstra

Para la ruta más corta entre un origen y un destino se usa el algoritmo de Dijkstra, que funciona porque todos los pesos (distancia o tiempo) son no negativos. En cada paso selecciona el nodo no procesado con menor distancia acumulada y relaja sus aristas salientes. La implementación permite elegir el peso (por tiempo o por distancia), respeta el sentido de las vías e ignora las aristas cerradas. Se reconstruye la ruta con un vector de predecesores y se imprime tramo por tramo con su costo y los totales. Se eligió la versión sencilla que busca el mínimo recorriendo el arreglo de distancias (sin cola de prioridad) por claridad; su costo es $O(V^2)$, suficiente para el tamaño de la red del campus.

5.5 Ruteo: A* con heurística euclidiana

A* es una mejora de Dijkstra que además de la distancia ya recorrida usa una estimación de lo que falta para llegar al destino. Aquí la estimación es la distancia euclidiana en línea recta entre las coordenadas del nodo actual y las del destino. Esa heurística es admisible (nunca sobreestima, porque la longitud real de un camino no puede ser menor que la distancia en línea recta), lo que garantiza que A* encuentre la ruta óptima. La ventaja es que A* tiende a expandir nodos en la

dirección del destino, evitando explorar zonas que se alejan, por lo que suele revisar menos nodos que Dijkstra para llegar al mismo resultado.

5.6 Ruta alternativa

La ruta alternativa responde a la pregunta: si el tramo más crítico de mi ruta principal estuviera fuera de servicio, ¿por dónde iría? Primero se calcula la ruta principal con Dijkstra. Luego se identifica en esa ruta la arista de mayor peso (la más crítica), se cierra temporalmente y se recalcula la ruta evitándola. Es la versión simple que pide el enunciado (recalcular evitando la arista más crítica de la ruta principal) y reutiliza el mismo Dijkstra, lo que mantiene el código compacto.

5.7 Árbol de expansión mínima (Kruskal + Union-Find)

Para el tendido mínimo que mantiene todo conectado al menor costo se usa el algoritmo de Kruskal. Se ordenan todas las aristas de menor a mayor longitud y se van tomando una por una, agregando cada arista solo si une dos grupos distintos (para no formar ciclos). La verificación de si dos nodos ya están en el mismo grupo se hace con la estructura Union-Find (conjuntos disjuntos), que ofrece las operaciones de encontrar el representante de un grupo y unir dos grupos. Se eligió Kruskal con Union-Find porque es la combinación clásica y directa para MST cuando se trabaja con la lista de aristas, y porque el enunciado lo solicita explícitamente. Al final se reporta el costo total del árbol, la lista de aristas seleccionadas y el porcentaje de ahorro respecto a usar la red completa.

5.8 Simulación de cierres

La simulación mide el impacto de deshabilitar una o varias conexiones (por mantenimiento o por un incidente). Primero se calcula la ruta entre origen y destino antes de cerrar nada; luego se marcan como cerradas las aristas indicadas y se recalcula la ruta; finalmente se reporta el desvío: la diferencia de longitud y de tiempo respecto a la situación original, o el aviso de que los nodos quedaron incomunicados si ya no existe ruta. Al terminar, las aristas se vuelven a abrir para no afectar otras consultas. Los algoritmos de ruteo ya ignoran las aristas marcadas como cerradas, por lo que la simulación reutiliza Dijkstra sin cambios.

5.9 Reportes (top-k)

Se incluyen dos reportes de elementos críticos. El primero ordena los nodos por su grado (cantidad de conexiones que los tocan) y muestra los k más conectados, que suelen ser los más importantes para la circulación. El segundo evalúa cada puente: lo cierra temporalmente, cuenta cuántos nodos quedan a cada lado y ordena los puentes por el tamaño del lado menor que aíslan; así se identifican los puentes cuyo fallo dejaría incomunicada la mayor cantidad de infraestructura. También se reportan la ruta óptima y la alternativa (secuencia de nodos con costo detallado) y el resumen del MST (costo y ahorro), cubriendo los reportes pedidos por el enunciado.

6. Formato de entrada y salida

Los datos de entrada son dos archivos CSV con encabezado. El formato sigue el ejemplo del enunciado.

vertices.csv:

```
id,name,x,y,type,priority
```

```
N001,Nodo 1,-2.161,-2.594,subestacion,alta
N002,Nodo 2,4.104,24.151,interseccion,baja
```

aristas.csv:

```
u,v,length,speed_limit,oneway,can_close
N001,N003,44.68,25,true,false
N001,N004,28.05,30,false,true
```

La salida es textual y se imprime en consola. Cada opción del menú produce un bloque con título y resultados: el resumen de validación, la representación textual del grafo (cada nodo con su tipo, prioridad y vecinos con largo y tiempo), las componentes, los puentes y articulaciones, las rutas paso a paso con sus costos y totales, el árbol de expansión mínima con su costo y ahorro, el desvío de la simulación y los reportes top-k.

7. Validación y manejo de errores

- Si un archivo CSV no se puede abrir, se informa el error y el programa termina de forma controlada.
- Se descartan e informan: líneas vacías, nodos sin id, nodos con id repetido y aristas cuyos extremos no existan.
- Antes de calcular una ruta se verifica que el nodo de origen y el de destino existan; si no, se muestra un mensaje y no se ejecuta el algoritmo.
- Si no existe ruta entre dos nodos (por ejemplo por el sentido de las vías o por cierres), se informa explícitamente en lugar de devolver un resultado incorrecto.
- En la división para el tiempo se protege el caso de velocidad cero para evitar una división entre cero.
- Las aristas cerradas durante la simulación se reabren al terminar, de modo que una consulta no contamina a la siguiente.
- Las opciones del menú fuera de rango muestran 'Opcion no valida' y vuelven a pedir.

8. Análisis de complejidad

En la siguiente tabla, V es el número de vértices (nodos) y E el número de aristas (conexiones). La implementación usa búsqueda lineal por id (buscarIndice, $O(V)$), decisión tomada por simplicidad; por eso varias operaciones incluyen un factor V adicional sobre la complejidad teórica del algoritmo. Para el tamaño de la red de un campus este sobre costo es perfectamente aceptable.

Módulo / Operación	Complejidad	Comentario
Buscar nodo por id (buscarIndice)	$O(V)$	Búsqueda lineal en el vector de nodos
Cargar vértices	$O(V^2)$	Verifica id repetido por cada nodo
Cargar aristas	$O(E * V)$	Ubica los extremos de cada arista
Componentes conexas (BFS / DFS)	$O(V + E) *$	Recorrido lineal del grafo
Puentes y articulaciones (Tarjan)	$O(V + E) *$	Un solo recorrido DFS
Dijkstra	$O(V^2)$	Selección del mínimo por arreglo
A*	$O(V^2)$	Dijkstra guiado por heurística
Ruta alternativa	$O(V^2)$	Dos ejecuciones de Dijkstra
MST (Kruskal + Union-Find)	$O(E \log E)$	Dominado por el ordenamiento de aristas
Simulación de cierres	$O(V^2)$	Cierra aristas y recalcula con Dijkstra

Módulo / Operación	Complejidad	Comentario
Reporte de nodos por grado	$O(E + V \log V)$	Calcula grados y ordena
Reporte de puentes por componentes	$O(P * (V + E))$	P = número de puentes; un BFS por puente

(*) El recorrido en sí es lineal $O(V + E)$; la construcción de la adyacencia no dirigida que usan estos módulos agrega un factor por la búsqueda de extremos. Las complejidades mostradas reflejan la implementación real, sencilla y legible, priorizada en este proyecto.

9. Conclusiones

El sistema demuestra cómo los algoritmos clásicos de grafos se integran de forma coherente para resolver un problema realista de inspección y mantenimiento de infraestructura. Cada algoritmo responde a una necesidad concreta del dominio: BFS y DFS para conocer la conectividad, Tarjan para hallar los elementos críticos, Dijkstra y A* para el ruteo, Kruskal con Union-Find para la cobertura mínima, y la simulación de cierres para cuantificar el impacto de un incidente.

La elección de cada estructura y algoritmo se justifica por el problema que resuelve: lista de adyacencia para una red dispersa, manejo del sentido de las vías en la propia adyacencia para que el ruteo sea correcto, heurística euclidiana admisible para que A* mantenga la optimalidad, y Union-Find para evitar ciclos en el MST de manera eficiente. La separación en clases (Grafo, UnionFind) y estructuras (Vertice, Arista), junto con la documentación interna, hace que el código sea claro y fácil de seguir.

Se priorizó de manera consciente una implementación sencilla y legible (por ejemplo, búsqueda lineal por id y selección del mínimo por arreglo en Dijkstra) por encima de optimizaciones más complejas, ya que para el tamaño de la red del campus el rendimiento es adecuado y la claridad del código es un objetivo del curso. Como mejoras futuras se podrían incorporar una tabla hash para localizar nodos en $O(1)$, una cola de prioridad para llevar Dijkstra a $O((V + E) \log V)$ y una representación gráfica del grafo mediante una herramienta como GraphViz.

Fin del documento técnico.